CPEN 416 / EECE 571O

Lecture 22: Fault-tolerant quantum computing

Thursday 27 November 2025

Announcements

- Presentations for next two classes - regular attendance expected
- Quiz 10 Tuesday before presentations
- Please fill out student experience survey on Canvas
- Exam information will be available in next few days; Monday tutorial, open to requests on what to review

Last time

We explored conditions for quantum error correction to work.

Theorem 10.1: (**Quantum error-correction conditions**) Let C be a quantum code, and let P be the projector onto C. Suppose $\mathcal{E}$ is a quantum operation with operation elements $\{E_i\}$. A necessary and sufficient condition for the existence of an error-correction operation $\mathcal{R}$ correcting $\mathcal{E}$ on C is that

(10.16)

for some Hermitian matrix α of complex numbers.

If such an $\mathcal{R}$ exists, $\{E_i\}$ is called a *correctable set of errors*.

Last time

We introduced the idea of *stabilizer codes*.

Let S be a subgroup of $\mathcal{P}_n$,

Let V_S be a set of states that are $+1$ eigenstates for all $P \in S$.

Then, S is the **stabilizer** of V_S, and V_S is stabilized by S.

Learning outcomes

1. Express the bit flip, phase flip, and Shor code in the stabilizer formalism
2. Derive parameters for the smallest code that corrects arbitrary single-qubit errors (and express it in the stabilizer formalism)
3. State the threshold theorem, and explain at a high level why error correction works even with faulty components

Bit flip code: stabilizers generators

Let's determine a *minimal* representation of the bit flip stabilizers in terms of its **stabilizer generators**.

Bit flip code: stabilizers and logical operations

Logical Z: need a Pauli that

- stabilizes the logical state but is *not* in S
- *commutes* with everything in S
- has the right action on the subspace

Logical X: need a Pauli that

- stabilizes the logical state but is *not* in S
- *commutes* with everything in S
- has the right action on the subspace
- anticommutes with logical Z

Bit flip code: stabilizer measurement and error detection

We can use this formalism to construct circuits for error detection and recovery: simply *measure the stabilizer generators*.

Bit flip code: stabilizer measurement and error detection

Correctible errors all anticommute with at least one generator. We can detect their presence in the syndrome measurement.

Bit flip code: stabilizers and logical errors

Danger: error that *commutes* with all elements of S, but isn't in S.

Phase flip code: stabilizer formalism

Shor code: stabilizer formalism

Name	Operator
g_1	$ZZIIIIIII$
g_2	$IZZIIIIII$
g_3	$IIIZZIIII$
g_4	$IIIIZZIII$
g_5	$IIIIIIZZI$
g_6	$IIIIIIIZZ$
g_7	$XXXXXXIII$
g_8	$IIIXXXXXX$
$\bar{Z}$	$XXXXXXXXX$
$\bar{X}$	$ZZZZZZZZZ$

Screenshot: Nielsen & Chuang, chapter 10.5.6.

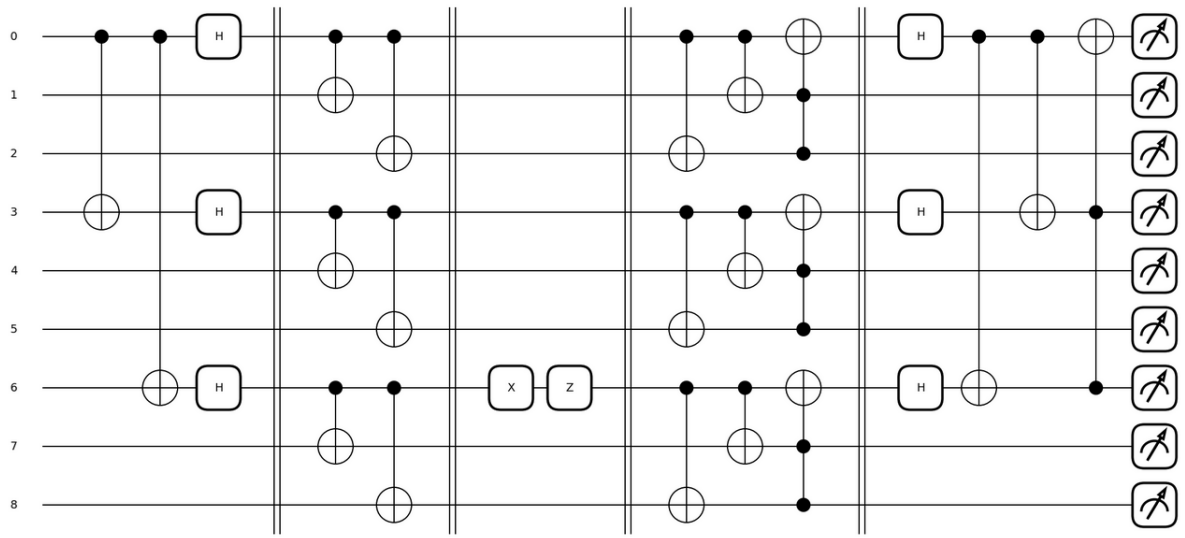
Properties of stabilizer codes

Stabilizer code usually described by notation $[[n, k, d]]$:

- code has $n - k$ stabilizer generators
- $d =$ distance (minimum weight of Paulis that commute with everything in S but aren't in S)
- a distance d code can correct $(d - 1)/2$ errors

Shor code

The [[9, 1, 3]] code can correct arbitrary single-qubit errors, but it uses 9 qubits. Can we do better?



Name	Operator
g_1	$ZZIIIIIII$
g_2	$IZZIIIIII$
g_3	$IIIZZIIII$
g_4	$IIIIZZIII$
g_5	$IIIIIIZZI$
g_6	$IIIIIIIZZ$
g_7	$XXXXXXIII$
g_8	$IIIXXXXXX$
$\bar{Z}$	$XXXXXXXXX$
$\bar{X}$	$ZZZZZZZZZ$

Screenshot: Nielsen & Chuang, chapter 10.5.6.

The Steane code: [[7, 1, 3]]

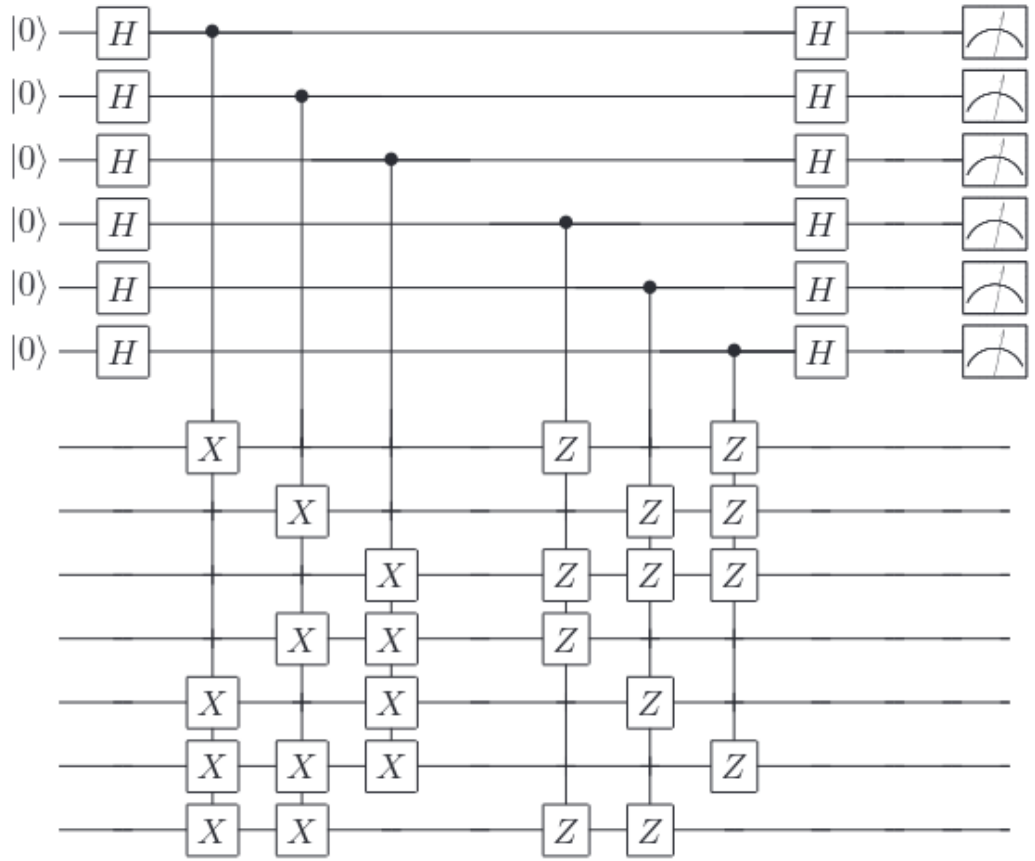


Figure 10.16. Quantum circuit for measuring the generators of the Steane code, to give the error syndrome. The top six qubits are the ancilla used for the measurement, and the bottom seven are the code qubits.

Screenshot: Nielsen & Chuang, chapter 10.5.8.

The Hamming bound

What is the smallest number of physical qubits we need for a logical qubit in a (non-degenerate) code that corrects arbitrary single-qubit errors?

General bound:

- n physical qubits
- k logical qubits
- up to t errors

The smallest code: [[5, 1, 3]]

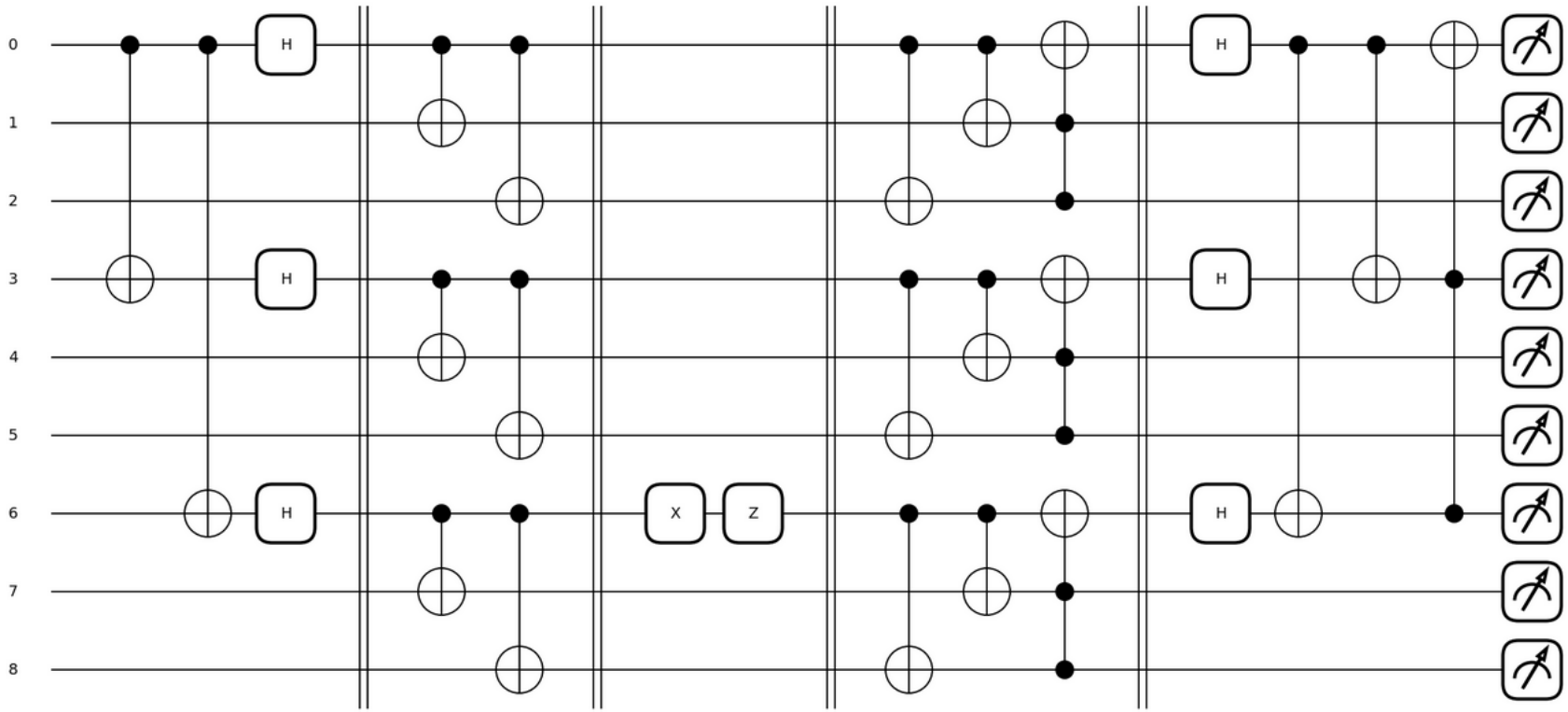
$$
\begin{aligned}
|0_L\rangle = \frac{1}{4}\Big[&|00000\rangle + |10010\rangle + |01001\rangle + |10100\rangle \\
&+|01010\rangle - |11011\rangle - |00110\rangle - |11000\rangle \\
&- |11101\rangle - |00011\rangle - |11110\rangle - |01111\rangle \\
&- |10001\rangle - |01100\rangle - |10111\rangle + |00101\rangle\Big] \\
|1_L\rangle = \frac{1}{4}\Big[&|11111\rangle + |01101\rangle + |10110\rangle + |01011\rangle \\
&+|10101\rangle - |00100\rangle - |11001\rangle - |00111\rangle \\
&- |00010\rangle - |11100\rangle - |00001\rangle - |10000\rangle \\
&- |01110\rangle - |10011\rangle - |01000\rangle + |11010\rangle\Big]
\end{aligned}
$$

Name	Operator
g_1	$XZZXI$
g_2	$IXZZX$
g_3	$XIXZZ$
g_4	$ZXIXZ$
$\bar{Z}$	$ZZZZZ$
$\bar{X}$	$XXXXX$

Screenshot: Nielsen & Chuang, chapter 10.5.6.

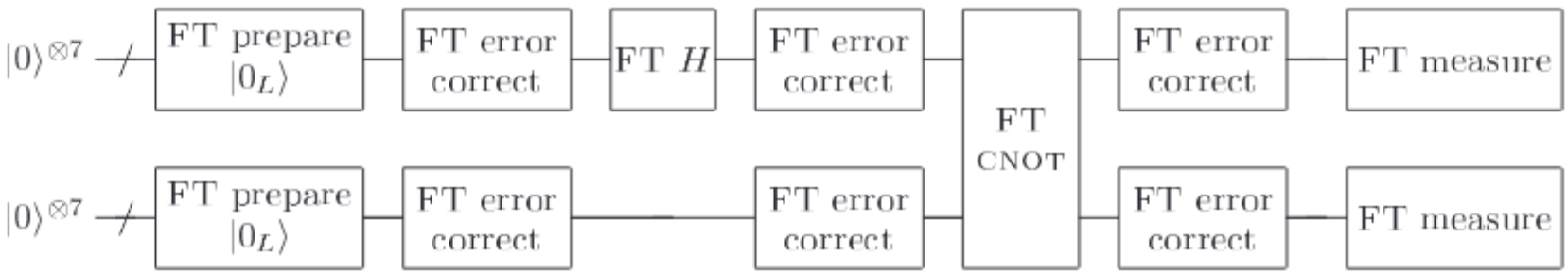
Fault-tolerant quantum computing

How can it be that doing all these extra operations is *better*, when in reality, all the operations are noisy?



Fault-tolerant quantum computing

Definition (adapted from Nielsen & Chuang): a procedure is *fault-tolerant* if the failure of a single component leads to *at most* one error per encoded block of qubits in the output.



If probability of error in any component is p, what is the probability of an error in the output? (and how do we ensure it's smaller?)

Fault-tolerant quantum computing

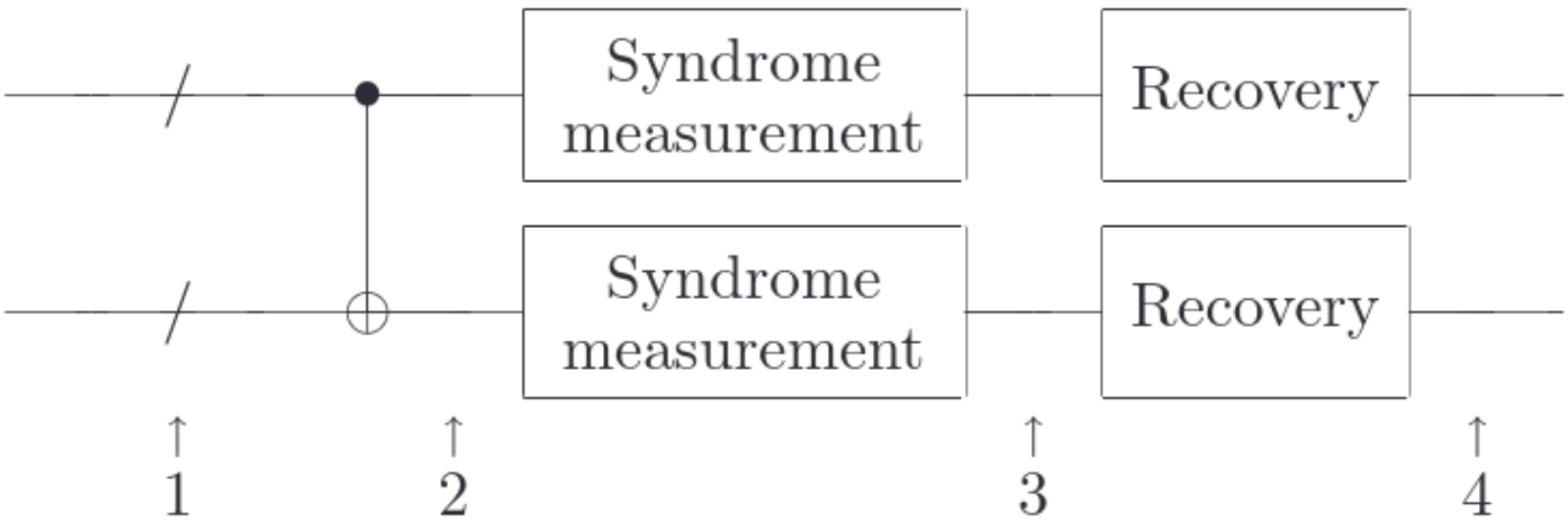
Need to define a noise model and consider how errors *propagate* through a circuit.

Assume noise consists of single-qubit Pauli errors with some associated probabilities (some correlation allowed).

Example:

Example: fault-tolerant CNOT

If one error occurs before/within this circuit, how many different ways can we experience *two or more* errors in the encoded first qubit at the output?



Screenshot: Nielsen & Chuang, chapter 10.6.1 (Fig. 10.21)

Example: fault-tolerant CNOT

We will use the Steane code as an example:

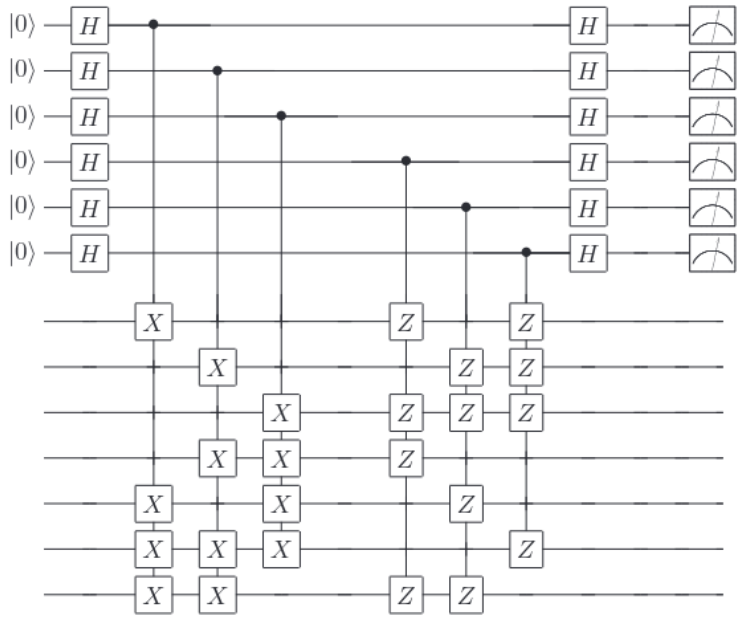


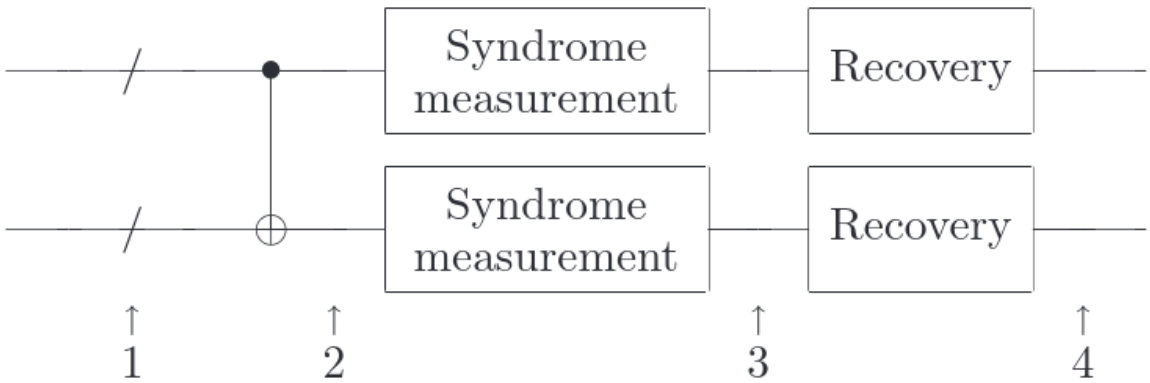
Figure 10.16. Quantum circuit for measuring the generators of the Steane code, to give the error syndrome. The top six qubits are the ancilla used for the measurement, and the bottom seven are the code qubits.

- 6 syndrome measurements (6 gates, measurement, recovery)
- logical CNOT is transversal (CNOT between all pairs); 7 points of failure that can propagate

Screenshot: Nielsen & Chuang, chapter 10.5.8.

Example: fault-tolerant CNOT

Example error mechanisms:



1. pre-existing error at step 1 on one qubit ($c_0 p$) or both qubits $c_0^2 p^2$, where $c_0 \approx 70$
2. two failures during the syndrome measurement $c_4 p^2$, where $c_4 \approx 70^2 \approx 5 \cdot 10^3$

Screenshot: Nielsen & Chuang, chapter 10.6.1 (Fig. 10.21)

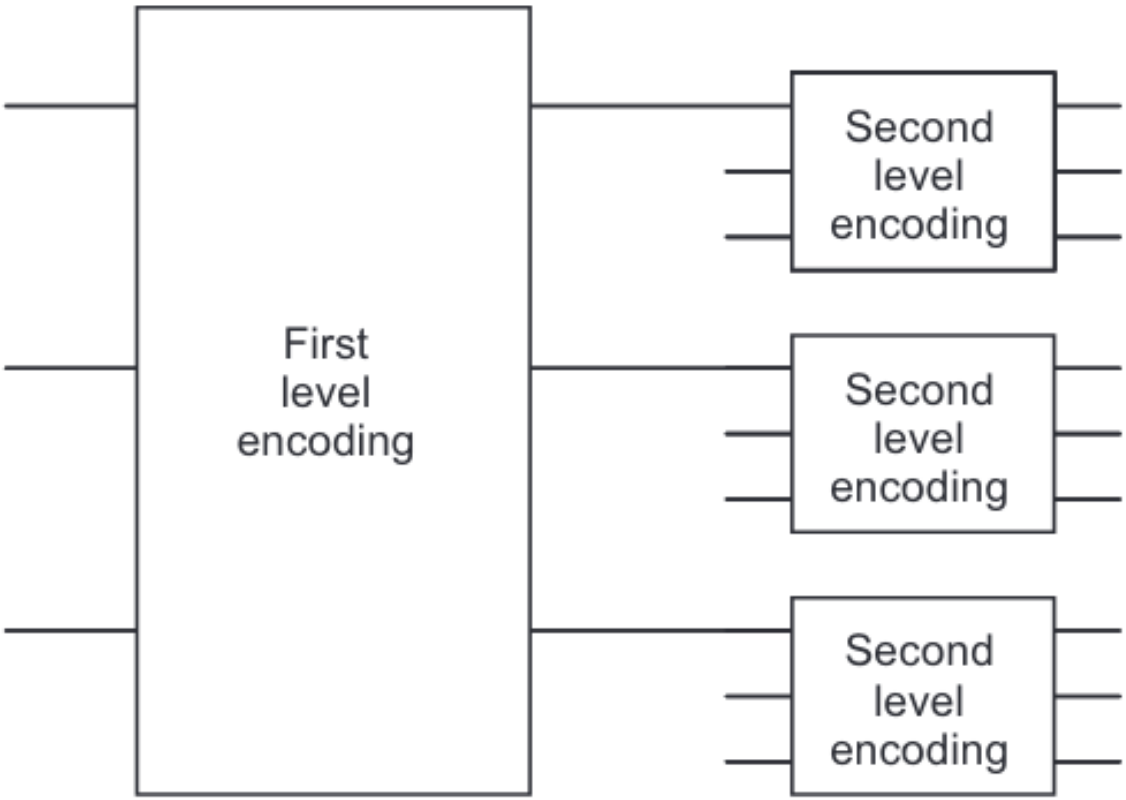
Example: fault-tolerant CNOT

Can work through example of Nielsen & Chuang; key point is that probability of two or more errors in first encoded block is

As long as , this is less than original error probability
(more robust estimates closer to)

The threshold theorem

What if our p is too large? Concatenate codes...



Screenshot: Nielsen & Chuang, chapter 10.6.1 (Fig. 10.22)

The threshold theorem

Q: if you concatenate k times, what is the failure probability at the k'th level?

Q: suppose g is the maximum number of operations in a fault-tolerant procedure. What is the simulation overhead?

Q: given a circuit with polynomial $p(n)$ gates, how many times do we need to concatenate to get total accuracy ϵ?

The threshold theorem

Can find k as long as $p < p_{th} = 1/c$ (*threshold condition*).

The size of the simulation circuit then becomes

Surface codes

Promising code, used in many recent demonstrations of fault-tolerance:

- threshold estimated at $\approx 1\%$
- logical error rate

$$\epsilon_d \propto \left(\frac{p}{p_{thr}}\right)^{\frac{d+1}{2}}$$

- suitable for nearest neighbour connectivity
- high qubit cost $(2d^2 - 1)$
- decoding is challenging

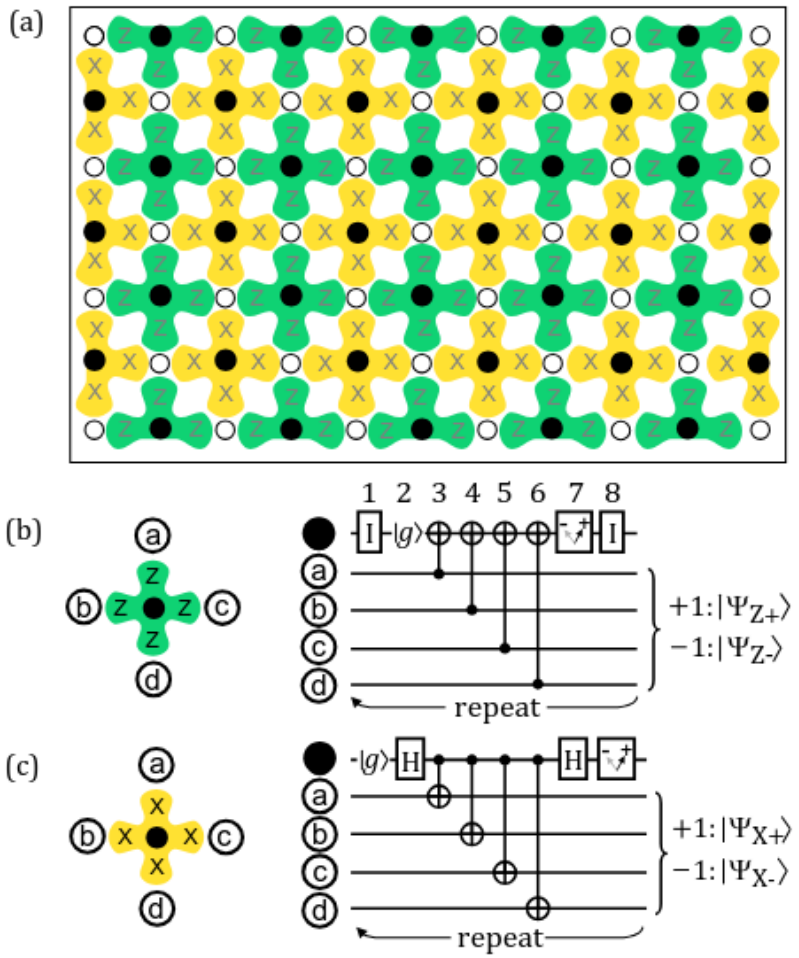


Image: Fowler et al. (2012) Surface codes: Towards practical large-scale quantum computation. PRA 86, 032324

Surface codes

In L01 I showed this plot of Google's *Willow* processor results. Now you can interpret it!

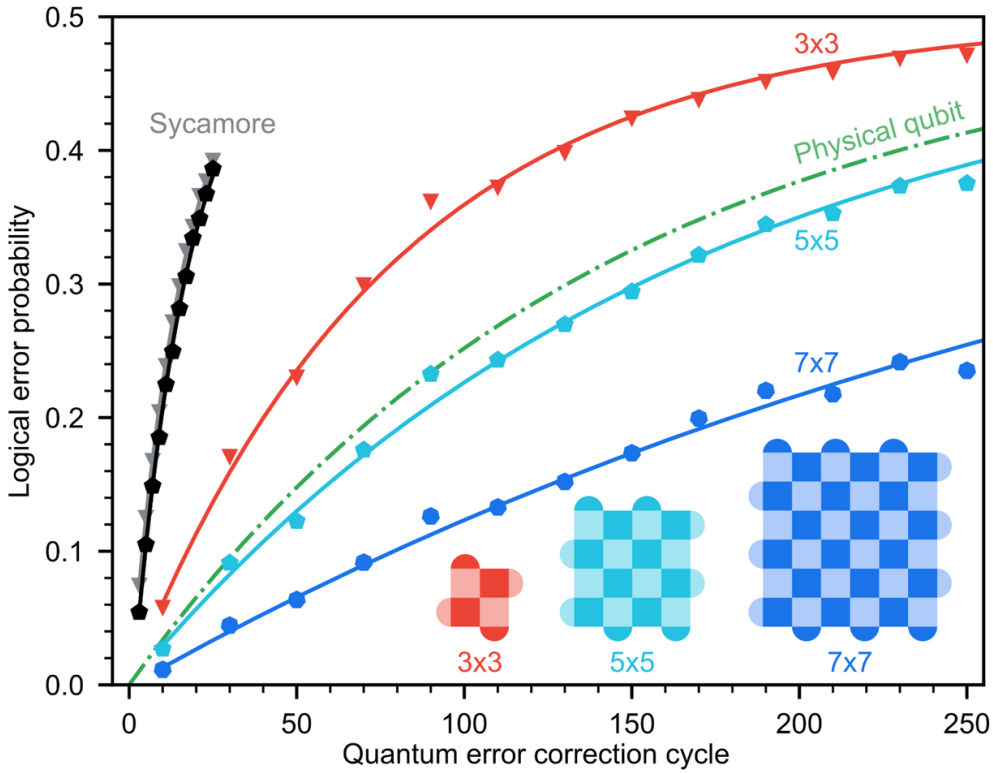


Image: https://research.google/blog/making-quantum-error-correction-work/

Open challenges in error correction

Very hot area, lots of open questions, and lots to do...

- find codes with better properties (fewer qubits, less connectivity limitations)
- improve performance of decoders
- develop the software stack (many compilation challenges!)
- continue to improve the hardware (better physical qubits means lower overhead)

Next time

Next class:

- Quiz 10
- Presentations

Action items:

1. Work on project
2. Check PrairieLearn for exam information (likely on weekend)

Recommended reading:

- For this class: Codebook EC; N&C 10.6